Тестове завдання: AI-first Backend Engineer

Про завдання

Зробіть невеликий бекенд-сервіс за описом нижче. Працюйте так, як працюєте зазвичай: з **Claude Code**, **Codex**, **Cursor** чи іншим агентом.

- **Орієнтовний час:** 3–4 години.

Задача

Сервіс обробляє звернення в підтримку: готує чернетку відповіді для оператора за допомогою LLM.

1. **Отримання:** Хелпдеск надсилає нове звернення на ваш вебхук.
2. **Генерація:** Сервіс знаходить релевантні статті в базі знань і генерує чернетку відповіді з посиланнями на них.
3. **Модерація:** Оператор у веб-інтерфейсі бачитиме список звернень із чернетками та може підтвердити або відхилити їх.
4. **Надсилання:** Підтверджена відповідь надсилається клієнту через API хелпдеску.

Інтерфейс має бути базовим, без уваги до дизайну (технології на ваш вибір). Базу знань згенеруйте за допомогою AI: 5–10 коротких статей про вигаданий продукт. Як їх зберігати й шукати — вирішуєте самі.

Умови

Справжнього хелпдеску немає, тому зробіть простий мок.

1. Хелпдеск

- **Вебхук:** POST /webhooks/helpdesk
- **Час очікування:** Хелпдеск чекає відповіді вебхука **не довше 2 секунд**, інакше повторює доставку.
- **Повтори та дублікати:** Та сама подія може прийти кілька разів, зокрема одночасно.
- **Навантаження:** Зазвичай надходить кілька звернень на хвилину, у піках — **до 500 за 10 хвилин**. Звернення не можна втрачати.

Формат тіла вебхука:

```JSON
{
  "event_id": "evt_8f2a",
```

```
  "ticket_id": "T-1042",
  "customer_email": "user@example.com",
  "message": "Не можу експортувати звіт у PDF",
  "reply_url": "https://helpdesk.example.com/tickets/T-1042/replies"
}
```

Відповідь клієнту:

- **Запит:** POST <reply_url>

```
JSON
{
  "ticket_id": "T-1042",
  "message": "Щоб експортувати звіт у PDF, відкрийте..."
}
```

- **Успішна відповідь хелпдеску:** 201 Created
- **Особливості:** Хелпдеск іноді повертає 5xx або не відповідає вчасно. Клієнт **не повинен отримати ту саму відповідь двічі**.

2. LLM

- **Провайдер і модель:** На ваш вибір.
- **Затримка:** Один виклик триває **5–30 секунд**.
- **Ліміти:** 60 запитів на хвилину на весь сервіс.
- **Режим без API-ключа:** Сервіс має працювати і без API-ключа: тоді він усе одно робить реальний запит до провайдера, а замість його відповіді використовує статичну із затримкою 5–30 секунд (як у реального виклику). Решта системи працює з нею так само, як із реальною відповіддю.

Стек

- **Мова/Фреймворк:** Python (**Django** або **FastAPI**)
- **База даних:** PostgreSQL
- **Запуск:** Через docker compose up
- *Решта інструментів — на ваш вибір.*

Що здати

- **Git-репозиторій** на GitHub з історією комітів «як є», **без squash**.
- **README:** детальна інструкція з запуску.
- **Конфігурація агента** (CLAUDE.md, rules, специфікації, документи), якщо використовувалися, — також мають бути закомічені в репозиторій.

- *(Дуже бажано)* Розгорнутий сервіс на будь-якому хостингу з публічними посиланнями на вебхук та інтерфейс оператора.